TITLE AND DATE

Document name: CMPE311 Project 4 (PROJECT-ADDMOTOR), Revision 4 of the CMPE311 Project 1 design document

Document reference: provost.cmpe311.fall26.project#4

Supersedes: provost.cmpe311.fall26.project#3 [3]

Change authority: ECO 0.3.1 [5]; notified by ECN 0.3.1 [4]

Date of publication: October 31, 2026

Lead engineer: Raheem Provost, UMBC

STAKEHOLDERS

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

REVISION HISTORY

Table 1. Revision history

Rev	Date	Author	Change
1	September 22, 2026	RP	Initial release (Project 1, PROJECT-ASYNC) [1]
2	October 3, 2026	RP	ECO 0.1 / ECN 0.1: task dispatch moved to a round-robin cyclic executive using a function pointer table; EC.6, T.7 added [2]
3	October 15, 2026	RP	ECO 0.3 / ECN 0.3: millis() replaced by mymilli() with its own Timer/Counter0 compare-match ISR and register constants (new files mymilli.h and mymilli.cpp); EC.7, EC.7.1, EC.7.2, T.8 to T.10 added; #include check of T.6 and T.7 changed [3]
4	October 31, 2026	RP	ECO 0.3.1 / ECN 0.3.1: PWM-driven DC motor fan with four speeds (OFF, LOW, MED, HIGH) and one push button added (new files fan.h, fan.cpp, button.h and button.cpp, Timer/Counter1, pins 9 and 10, a fourth task in the task table); HL.7 to HL.18, EC.8, EC.9 and T.11 to T.14 added; T.6, T.7, T.8 and T.10 changed

HIGH-LEVEL DESCRIPTION

This document is the project report for Project #4 of the CMPE311 class. It is Revision 4 of the Project #1 design document [1] and records the design change required by ECO 0.3.1 [5]. Sections that did not change are cited to [3], [2] and [1] rather than restated.

DESCRIPTION

This design implements an embedded system on an Arduino Uno R3 development platform that independently controls the blink rate of two LEDs. The User interacts with the System through the Arduino IDE serial monitor, specifying an LED number and a blink interval in milliseconds. In this revision the System also drives a DC motor fan. The speed of the fan has four states (OFF, LOW, MED and HIGH) and is set by a

single push button, which steps the fan through OFF, LOW, MED, HIGH, MED, LOW, OFF and then repeats. The fan is driven by a PWM signal made by Timer/Counter1. The LED behavior seen by The User is identical to Project #3.

RESULT SUMMARY

The embedded system design adds a PWM-driven DC motor fan with four speeds, controlled by one push button, using its own Timer/Counter1 Fast PWM setup and register constants in the fan module (fan.h, fan.cpp) and its own debounced button read in the button module (button.h, button.cpp). The results of the tests are to be recorded from the demonstration and from inspection of the final source files; see TESTING RESULTS. The LED behavior seen by The User is identical to Project #3.

REFERENCES AND GLOSSARY

REFERENCES

1. provost.cmpe311.fall26.project#1, CMPE311 Project 1 design document (PROJECT-ASYNC), Revision 1, R. Provost, September 22, 2026.
2. provost.cmpe311.fall26.project#2, CMPE311 Project 2 design document (PROJECT-CE), Revision 2, R. Provost, October 3, 2026.
3. provost.cmpe311.fall26.project#3, CMPE311 Project 3 design document (PROJECT-MYMILLI), Revision 3, R. Provost, October 15, 2026.
4. provost.cmpe311.fall26.project#4.ECN, Engineering Change Note ECN 0.3.1, R. Provost, October 31, 2026.
5. PROJECT: PROJECT-ADDMOTOR assignment, including Engineering Change Order ECO 0.3.1 (3/1/2030, revised 4/1/2030, requestor Prof. Kidd), CMPE311.
6. Microchip Technology, ATmega328P Data Sheet: 16-bit Timer/Counter1 with PWM (Table 15-5, waveform generation modes), I/O ports and register summary.

DEFINITIONS

The definitions of The User, The System, The Customer, The Developer, The Evaluator, The Customer-requirements, The HL-requirements, The Requirements, The Educational-constraints, The Company, The Contract, serial-monitor and The Reference-platform are identical to those of PROJECT-ASYNC. See [1] REFERENCES AND GLOSSARY section, DEFINITIONS. The terms added in Revision 2 (cyclic executive, round-robin, task, task table, dispatch round) are unchanged; see [2]. The terms added in Revision 3 (arduino.h, avr.h, millis(), mymilli(), mymilli.h, mymilli.cpp, time base, tick) are unchanged; see [3]. The following are added in this revision:

"Fan" – The DC motor with fan blade that moves air in the System.

"Fan state" – One of OFF, LOW, MED or HIGH, the four speeds of the fan.

"Push button" – The single-pole, momentary-on, normally-off switch that sets the fan state.

"Duty cycle" – The fraction of each PWM period during which the PWM output is high.

"Debounce" – Ignoring the short, repeated contact changes of a push button just after it is pressed or released.

"Fan module" – The files fan.h and fan.cpp, written by The Developer. They set up the fan PWM and hold the fan state machine.

"Button module" – The files button.h and button.cpp, written by The Developer. They read the push button, debounce it and report each release.

ACRONYMS AND ABBREVIATIONS

The acronyms and abbreviations of [1], [2] and [3] REFERENCES AND GLOSSARY sections are unchanged. The following are added in this revision:

CFM – Cubic Feet per Minute

dB – Decibel

DC – Direct Current

MOSFET – Metal-Oxide-Semiconductor Field-Effect Transistor

OC1A – Timer/Counter1 output compare A pin (digital pin 9)

PROJECT-ADDMOTOR – Project #4, documented in this document

RPM – Revolutions Per Minute

TOP – The counter value at which Timer/Counter1 restarts its count

REQUIREMENTS

CONVENTIONS

The conventions are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, CONVENTIONS. Requirements added after Revision 2 are tagged with the date and the author's initials, e.g. "HL.7 (added 10/31/26 – RP)". Deleted text is ~~struck out~~ and carries its date.

PART I – CUSTOMER REQUIREMENTS AND DERIVED HIGH-LEVEL REQUIREMENTS

The customer requirements C1 through C6 of PROJECT-ASYNC are identical. See [1] REQUIREMENTS section, PART I, Table 1. ECO 0.3.1 adds seven customer requirements, numbered C6 through C12 in Table 1 of [5]. They are for reference only. The high-level technical requirements derived from them are HL.7 through HL.18 (Table 2): a motor-driven fan (C6, HL.7); RPM controlled by a push button (C7, HL.12); four speeds (C8, HL.14); each button hit cycles the speeds (C9, HL.15); and the sound and air flow of LOW, MEDIUM and HIGH (C10 to C12, HL.16 to HL.18). The ECO's C6 is distinct from C6 of [1].

PART II – HIGH-LEVEL REQUIREMENTS AND EDUCATIONAL CONSTRAINTS

The high-level technical requirements, HL.1 through HL.6, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART II, Table 3. The following high-level technical requirements are added to implement ECO 0.3.1 [5]:

Table 2. High-level technical requirements added in this revision

ID	High-level technical requirement
HL.7 (added 10/31/26 – RP)	The device must have a DC motor-driven fan with an operating voltage between 3 V and 12 V.
HL.8 (added 10/31/26 – RP)	The fan must operate at a minimum between 1500 and 6500 RPM.
HL.9 (added 10/31/26 – RP)	The fan must draw less than 4 A.
HL.10 (added 10/31/26 – RP)	The fan must be driven by a PWM signal with a maximum voltage of 5 VDC.
HL.11 (added 10/31/26 – RP)	The fan must have a maximum dimension less than 4 in x 4 in x 3 in (not including cage).
HL.12 (added 10/31/26 – RP)	The RPM of the motor must be controllable using a single-pole, momentary-on, normally-off push button.
HL.13 (added 10/31/26 – RP)	The operation of the button and motor must have no noticeable effect upon the operation of previously implemented I/O (e.g. serial monitor and LEDs).
HL.14 (added 10/31/26 – RP)	The fan must have 4 speeds: OFF, LOW, MEDIUM and HIGH.
HL.15 (added 10/31/26 – RP)	Each hit of the button must cycle the speeds from OFF to LOW to MEDIUM to HIGH to MEDIUM to LOW to OFF.

ID	High-level technical requirement
HL.16 (added 10/31/26 – RP)	LOW must be able to move air with a sound volume less than 35 dB and airflow levels between ~~30-45~~ (deleted 4/1/2030 twk) 0.30 to 0.45 CFM ± 0.05 CFM.
HL.17 (added 10/31/26 – RP)	MEDIUM must be able to move air with a sound volume less than 40 dB and airflow levels between ~~45-70~~ (deleted 4/1/2030 twk) 0.45 to 0.70 CFM ± 0.05 CFM.
HL.18 (added 10/31/26 – RP)	HIGH must be able to move air with a sound volume less than 50 dB and airflow levels between ~~70-100~~ (deleted 4/1/2030 twk) 0.70 to 1.00 CFM ± 0.05 CFM.

The educational constraints, EC.1 through EC.5 including EC.2.1, are identical to those of PROJECT-ASYNC. See [1] REQUIREMENTS section, PART II, Table 4. EC.6 is unchanged from PROJECT-CE; see [2] REQUIREMENTS section, Table 2. EC.7, EC.7.1 and EC.7.2 are unchanged from PROJECT-MYMILLI; see [3] REQUIREMENTS section, Table 2. The following educational constraints are added to implement ECO 0.3.1 [5]:

Table 3. Educational constraints added in this revision

ID	Educational constraint	Source
EC.8 (added 10/31/26 – RP)	The Evaluator must be able to successfully have The System perform the dialog in Table 4 and Table 5 of [5].	ECO 0.3.1, EC 1.2 (E.4, added 3/20/2030 – kidd) [5]
EC.9 (added 10/31/26 – RP)	H.16, H.17 and H.18 do not have to be explicitly measured. Just provide some evidence that these requirements have been satisfied through justified inference such as through calculations.	ECO 0.3.1, EC 1.3 (E.5) [5]

//Informative: ECO 0.3.1 labels these two constraints EC 1.2 and EC 1.3 (E.4 and E.5 in its Table 3). They are numbered EC.8 and EC.9 here because EC.4 and EC.5 already exist. The earlier wording of E.4, which named only Table 5, was deleted 3/20/2030 (twk). Table 4 of [5] is the LED dialog and Table 5 of [5] is the button and fan sequence. The other items of ECO 0.3.1 are met as shown in Table 4.//

Table 4. How each item of ECO 0.3.1 is met

ECO item	Requirement	Where it is met
Description of change	Add a motor driving a fan with 4 states, its speed set by a push button	HL.7 to HL.18; HARDWARE DESIGN; SOFTWARE DESIGN
Description of change	Change the Design Documents of PROJECT-MYMILLI	This document (Revision 4); ECN 0.3.1 [4]
EC 1.2 (E.4)	The Evaluator can have The System perform the dialogs of Table 4 and Table 5 of [5]	EC.8; T.12 and T.13
EC 1.3 (E.5)	H.16, H.17 and H.18 justified by inference such as calculations	EC.9; T.14

DESIGN

DESIGN PRE-REQUISITES

The design pre-requisites are identical to those of PROJECT-ASYNC. See [1] DESIGN section, DESIGN PRE-REQUISITES.

BILL OF MATERIALS (CHANGED)

The bill of materials of PROJECT-ASYNC is unchanged and is cited to [1] DESIGN section, Table 5. The following items are added in this revision:

Table 5. Bill of materials: items added in this revision

Qty	Item	Use
1	DC motor with fan blade (Arduino kit), rated 3 V to 6 V	Fan
1	N-channel MOSFET, IRF520N	Low-side switch for the motor
1	330 Ω resistor	Gate resistor
1	10 kΩ resistor	Gate pull-down
1	Schottky diode	Flyback protection across the motor
1	Momentary tactile push button	Fan speed control
1	Breadboard power module (5 V output) and 9 V battery	Motor supply

DEVELOPMENT PLATFORM

The development platform is identical to that of PROJECT-ASYNC: Arduino IDE 2.3.10 on Windows 11, with the AVR-GCC toolchain bundled in the Arduino AVR Boards package 1.8.7 (EC.1). See [1] DESIGN section, DEVELOPMENT PLATFORM.

SYSTEM OVERVIEW (CHANGED)

The User still interacts with the System through the Arduino IDE serial monitor for the LEDs, with the same prompts as PROJECT-ASYNC. In this revision The User also presses a push button to change the speed of the fan. The work is now split into four tasks: the three tasks of PROJECT-MYMILLI (`askQuestions()`, `serviceLED1()`, `serviceLED2()`) and a new task, `checkButton()`, dispatched one per pass of `loop()` from `taskTable[]`. See [3] DESIGN section, SYSTEM OVERVIEW and [2] Table 3.

What changed is that the System now reads a push button and drives a fan. The new task passes each button release to the fan module, which moves the fan one step. The PWM signal that drives the fan is made by Timer/Counter1 in hardware, so the processor only writes one register when the fan state changes. The `mymilli()` time base is unchanged; the button debounce reads it. Figure 1 shows the path from the button to the fan.

HARDWARE DESIGN (CHANGED)

The LED hardware is identical to that of PROJECT-ASYNC: LED1 on digital pin 2 and LED2 on digital pin 3, each through a 330 Ω current-limiting resistor to a common ground. See [1] DESIGN section, HARDWARE DESIGN, Figure 1 (Schematic) and Table 6 (Arduino pin assignments). Timer/Counter0 is internal; none of its output pins (digital pins 5 and 6) is connected to the System. The following hardware is added in this revision.

Fan and driver. The fan is a small brushed DC motor with a fan blade. An Arduino pin cannot supply motor current, so an N-channel MOSFET switches the motor on the ground side (low-side switch). Digital pin 9 drives the MOSFET gate through a 330 Ω resistor that limits gate current. A 10 kΩ resistor from the gate to ground holds the motor off while pin 9 is not driven. A Schottky diode across the motor, with its cathode (the band) on the positive supply, absorbs the voltage spike when the motor current is switched off. The MOSFET is an IRF520N. It is not a logic-level part; it was tested with the gate held at 5 V and runs this motor (see DESIGN RATIONALE AND ASSUMPTIONS).

Supply. The motor runs from the 5 V output of a breadboard power module fed by a 9 V battery. The Arduino is powered through USB, as in [1]. The Arduino ground, the power module ground and the MOSFET source are joined.

Push button. A single-pole, momentary-on, normally-off tactile push button connects digital pin 10 to ground. The internal pull-up of pin 10 is turned on, so the pin reads high while the button is released and low while it is pressed. No external resistor is used.

Table 6. Arduino pin assignments added in this revision

Arduino pin	Port bit	Use	Status
9	PB1 (OC1A)	Fan PWM output to the MOSFET gate	Added 10/31/26 – RP
10	PB2	Push button input (internal pull-up)	Added 10/31/26 – RP

Pins 2 and 3 (LED1 and LED2) and the serial pins are unchanged; see [1] Table 6.

Table 7. Connections added in this revision

From	To
Arduino pin 9	330 Ω resistor, then the MOSFET gate
MOSFET gate	10 kΩ resistor to ground
MOSFET source	Ground
MOSFET drain	Motor negative terminal
Motor positive terminal	+5 V of the power module
Schottky diode	Across the motor: cathode (band) to +5 V, anode to the MOSFET drain
Push button	One side to Arduino pin 10, the other side to ground
Grounds	Arduino GND, power module ground and MOSFET source joined

SOFTWARE DESIGN (CHANGED)

The program keeps the PROJECT-MYMILLI task table, dispatcher, serial dialog, blink logic and time base. See [3] DESIGN section, SOFTWARE DESIGN. The changes are two new modules, the fan module (`fan.h`, `fan.cpp`) and the button module (`button.h`, `button.cpp`); the added `#include` lines at the top of the sketch for their headers; the added start-up calls at the end of `setup()`, after `mymilliInit()`; and one added task, `checkButton()`, the fourth entry of `taskTable[]`. The dispatcher in `loop()` is unchanged because it counts the table entries itself (`NUM_TASKS`).

Modules. The button module reads the push button, debounces it and reports one event each time the button is released. The fan module sets up Timer/Counter1 and holds the fan state machine, which moves one step on each button event. Keeping them apart lets the button be reused for another job and the fan be driven from another input.

Timer selection. The fan PWM uses Timer/Counter1, a 16-bit timer. Timer/Counter0 belongs to `mymilli()`, and Timer/Counter2 has its output on digital pin 3, which is LED2. Timer/Counter1 has its OC1A output on digital pin 9, so the hardware makes the PWM with no interrupt and no software timing. The Arduino core does not use Timer/Counter1 for `millis()`, so there is no conflict with the time base.

Timer configuration. Timer/Counter1 is set to Fast PWM mode 14 (waveform generation bits WGM13:0 = 0b1110), in which the counter counts from 0 up to TOP, held in ICR1, and starts again [6]. The compare output A is non-inverting (COM1A1:0 = 0b10): OC1A is set at BOTTOM and cleared when the counter matches OCR1A. TOP is 249 and the clock is divided by 8 (clock select bits CS12:0 = 0b010). At 16 MHz the timer counts every 0.5 µs, and one period is 250 counts, or 125 µs, which is 8 kHz. Table 8 gives the parameters.

Table 8. Timer/Counter1 parameters

Parameter	Value	Basis
System clock	16,000,000 Hz	Reference platform
Timer/Counter1 mode	Fast PWM, TOP = ICR1 (WGM13:0 = 0b1110)	Mode 14 [6]
Output	OC1A, non-inverting (COM1A1:0 = 0b10), digital pin 9 (PB1)	Set at BOTTOM, cleared on match [6]
Clock select	System clock / 8 (0b010)	2 MHz, 0.5 µs per count
TOP (ICR1)	249 (0b0000000011111001)	Counts 0 to 249
PWM period and frequency	250 counts = 125 µs = 8 kHz	TOP + 1 counts of 0.5 µs
Duty compare (OCR1A)	0, 100, 160, 249	See Table 10
OCR1A update	At BOTTOM	OCR1A is double buffered in mode 14 [6]

Register constants. The button and fan modules define the addresses of the Port B registers and the Timer/Counter1 registers they use, and the position of the clock select field, under names of their own, in the same way as `mymilli.cpp` (EC.7.2). Table 9 maps each constant to the register or bit it stands for in [6]. The

modules include only `<stdint.h>` for the standard integer types; no AVR header is included, and no AVR-defined name for a register or a register bit is used.

Table 9. Register constants defined in the button and fan modules

Constant	Address / bit	Hardware item it stands for	Use
`myPINB`	0x23	Port B input pins	Read the push button on bit 2 (pin 10)
`myDDRB`	0x24	Port B data direction register	Bit 1 output (fan, pin 9); bit 2 input (button, pin 10)
`myPORTB`	0x25	Port B data register	Turn on the pull-up of bit 2
`myTCCR1A`	0x80	Timer/Counter1 control register A	Select non-inverting output and the low bits of mode 14
`myTCCR1B`	0x81	Timer/Counter1 control register B	Select the high bits of mode 14 and the clock (divide by 8)
`myICR1`	0x86	Timer/Counter1 input capture register (16-bit)	Hold TOP, 249
`myOCR1A`	0x88	Timer/Counter1 output compare register A (16-bit)	Hold the duty compare value
`clksel`	bit 0	Position of the clock select field (CS12:0) in control register B	Shift for the clock select value

Fan states and duty cycle. The fan has four states. Each state sets `OCR1A` to one of four constants, and the duty cycle is about `OCR1A` ÷ (TOP + 1), accurate to one count (0.4%). HIGH sets `OCR1A` equal to TOP; in non-inverting mode the compare match is then ignored and the pin stays high [6]. OFF sets `OCR1A` to 0, which leaves a pulse one count (0.5 µs) wide every period, a 0.4% duty that is too small to turn the motor. On the 5 V supply the average voltage on the motor is the duty cycle times 5 V.

Table 10. Fan states and duty cycles

State	OCR1A	Duty cycle	Average motor voltage
OFF	0	0%	0 V
LOW	100	40%	2.0 V
MED	160	64%	3.2 V
HIGH	249 (equal to TOP)	100%	5.0 V

Button read and debounce. The button module reads bit 2 of `myPINB`. With the pull-up on, the pin reads 1 at rest and 0 while the button is pressed. The module remembers the last accepted level and accepts a change only if at least 30 ms (30 ticks of `mymilli()`) have passed since the last accepted change, so the short contact bounce is ignored. It reports an event exactly once per release (pressed to released). A press is tracked but not reported.

State machine. The fan module reads the button event once per call, so every state sees the same event, and then runs a switch on the current state. A second variable remembers the state the fan came from. LOW and MED each occur twice in the cycle, once going up and once going down, and that variable tells the two visits apart. Each button release moves the fan one step, as Table 11 shows, and the cycle repeats.

Table 11. State transitions (one step per button release)

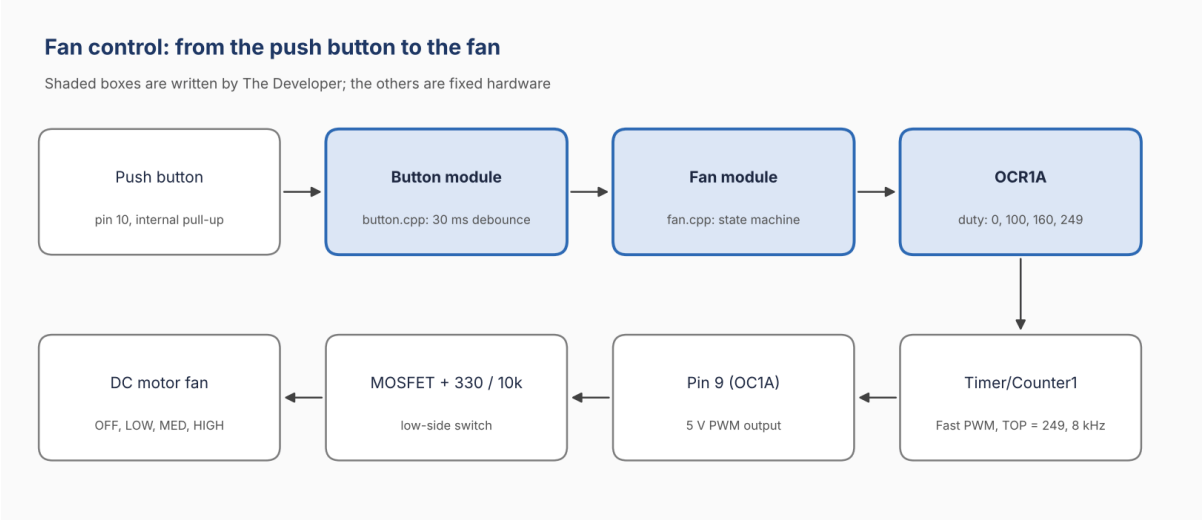
Current state	Came from	Next state	`OCR1A` set to
OFF	any	LOW	100
LOW	OFF (going up)	MED	160
MED	LOW (going up)	HIGH	249
HIGH	MED	MED	160
MED	HIGH (going down)	LOW	100
LOW	MED (going down)	OFF	0

Start-up. `setup()` configures pins 2 and 3 and the serial port, then calls `mymilliInit()`, then starts the button and fan modules. The order matters because the button debounce reads `mymilli()`. Start-up turns on the pull-up of pin 10, makes pin 9 an output and pin 10 an input, programs Timer/Counter1, sets `OCR1A` to 0, and writes control register B last, so the timer starts only after the setup is complete. The fan starts OFF.

LED timing. Unchanged from [3]. Each LED keeps its own last-toggle time and toggles when `mymilli()` reaches that time plus its interval. The fan adds no interrupt.

Arduino timing and PWM functions. As in [3], `millis()`, `micros()` and `delay()` do not work in this sketch, and `analogWrite()` on pins 5 and 6 is unavailable. In this revision `analogWrite()` on pins 9 and 10 (the Timer/Counter1 outputs) and the Servo library are also unavailable. The System uses none of them (T.8). EC.2 is still met because no Arduino core file is edited.

Figure 1. Fan control data flow (from the push button to the fan)



Dispatch logic is unchanged from PROJECT-CE except for the added fourth entry of the task table; see [2] DESIGN section, Figure 1. The time base data flow is unchanged; see [3] Figure 1.

Repository URL: https://github.com/raheemprovost/CMPE311Project4

Build and run steps are identical to those of PROJECT-ASYNC: open the `.ino` file in Arduino IDE 2.3.3 or later (`mymilli.h`, `mymilli.cpp`, `fan.h`, `fan.cpp`, `button.h` and `button.cpp` sit in the same folder as the `.ino` file and are built with it), select Arduino Uno and its port, upload, and open the Serial Monitor at 9600 baud with the line ending set to "Newline". The fan circuit of HARDWARE DESIGN is built before the button is used.

DESIGN RATIONALE AND ASSUMPTIONS (CHANGED)

- **One task per pass.** Calling one task per `loop()` pass gives every task the same bounded turn and keeps the dispatcher free of task-specific code. A new task is one function plus one table entry, which is how the button and fan were added. Unchanged in this revision.
- **Timing still holds.** Each task now runs once per four passes. Every task is a few comparisons and at most one `digitalWrite()`, `Serial.read()` or register read, so a full dispatch round takes tens of microseconds (calculated, not measured), well under the 100 ms interval. The compare-match ISR adds one short interruption per millisecond, and the fan adds none.
- **Serial input keeps up.** At 9600 baud a character arrives about every 1.04 ms. The serial task runs far more often than that, and the Arduino's 64-byte receive buffer holds any burst, so no input is lost. The compare-match ISR runs for a few microseconds 1,000 times a second, an estimated load of well under 1% (calculated, not measured). The PWM is made by hardware and adds no load.
- **Time base choices.** Why Timer/Counter0 in CTC mode, the resolution, accuracy, interval range, invalid input handling and run time are unchanged from [3]. The `mymilli()` comparison is not rollover-safe, so continuous operation is supported for up to about 49.7 days.

- **Why Timer/Counter1 and pin 9.** Timer/Counter0 is reserved for `mymilli()`, and Timer/Counter2 has its output on pin 3, which is LED2. Pin 9 is the OC1A output of Timer/Counter1, so the hardware makes the PWM with no interrupt. Pins 5 and 6 (the Timer/Counter0 outputs) are avoided.
- **Why Fast PWM at 8 kHz.** Fast PWM is the simplest mode to set up and reaches a high frequency. At 8 kHz the motor current is smooth, and the PWM is less likely to cause a low-frequency hum than at 1 kHz or below, which helps keep the sound level down (HL.16 to HL.18). A divide-by-8 clock with TOP = 249 gives exactly 8 kHz.
- **Why HIGH is OCR1A = TOP.** The data sheet states that the compare match is ignored when OCR1A equals TOP in non-inverting mode, so the pin stays high and HIGH is a true 100% duty with no glitch [6].
- **Why the fan changes state on a release.** One press is one step because the button reports only the release. Holding the button does not step the fan again, and a press that bounces does not step it twice.
- **Why the button is polled, with a 30 ms debounce.** Polling keeps the design to one more task in the existing task table and adds no interrupt. A 30 ms lockout, measured with `mymilli()`, ignores contact bounce, which is much shorter, while a person cannot press and release the button in less than that time.
- **No noticeable effect on the LEDs and serial dialog (HL.13).** The PWM is made by Timer/Counter1 hardware, so the processor only writes `OCR1A` when the state changes. The button is polled once per `loop()` pass by code that never waits. No interrupt is added, and Timer/Counter0 and `mymilli()` are untouched. Pins 9 and 10 are not shared with the LED pins or the serial pins.
- **Why a MOSFET, and why the IRF520N.** A pin cannot supply motor current, so a low-side N-channel MOSFET switches the motor. The IRF520N is not a logic-level part. It was tested with its gate held at 5 V and runs this motor, so it was kept. A logic-level part such as the IRLZ44N turns on fully at 5 V and can be fitted in its place.
- **Resources given up.** Timer/Counter0 is reserved for `mymilli()` and Timer/Counter1 for the fan PWM. The Arduino `millis()`, `micros()` and `delay()`, `analogWrite()` on pins 5, 6, 9 and 10, the Servo library, and libraries that depend on them, cannot be used. The System uses none of them. Pins 2 and 3 are plain digital outputs and are not affected.
- **Header includes.** The sketch includes only headers written by The Developer. No library or scheduler is included, so EC.5 still holds. See TEST PROCEDURES for how T.6 and T.7 treat them.
- **Assumptions for the EC.9 calculations.** (1) Motor speed and air flow scale with the average motor voltage, which is the duty cycle times the 5 V supply (first-order DC motor and fan behavior). (2) At HIGH the fan moves between 0.70 and 1.00 CFM and is quieter than 50 dB; no air flow or sound equipment is available, and EC.9 allows inference. (3) The 5 V power module limits the motor current to about 700 mA. (4) The clock is 16 MHz.

TESTING AND VALIDATION

DESCRIPTION

All PROJECT-MYMILLI tests, T.1 through T.10, are re-run on the PROJECT-ADDMOTOR build to show that adding the fan did not alter the earlier behavior of the System (HL.13). The #include check inside T.6 and T.7 and the file lists of T.8 and T.10 are changed, as described under TEST PROCEDURES. T.11 through T.14 are added to verify the fan (ECO 0.3.1, EC 1.2 and EC 1.3). Results are in Table 17 and evidence is indexed in Appendix A.

TESTING PLATFORM

The testing platform is identical to that of PROJECT-ASYNC: ELEGOO Arduino Uno R3, Arduino IDE 2.3.10, powered through USB, LEDs on digital pins 2 and 3 through 330 Ω resistors. See [1] TESTING AND VALIDATION section, TESTING PLATFORM. The fan circuit of HARDWARE DESIGN is added on the breadboard: the motor runs from the 5 V power module and 9 V battery, and the push button is on pin 10.

REQUIRED TEST DIALOG

The required LED test dialog is identical to that of PROJECT-ASYNC. See [1] TESTING AND VALIDATION section, Table 7. EC.4 and EC.8 still require the System to perform it. EC.8 also requires the System to perform the button and fan sequence of Table 5 of [5], which is added in this revision:

Table 12. Required fan test sequence (from Table 5 of [5])

Button	Fan
Initial state: button un-pressed	Initial state: fan off
Press	Fan on low
Press	Fan on medium
Press	Fan on high
Press	Fan on medium
Press	Fan on low
Press	Fan off
(repeat sequence above)	

TESTING AND VALIDATION REQUIREMENTS

The testing requirements T.1 through T.5 are identical to those of PROJECT-ASYNC. See [1] TESTING AND VALIDATION section, Table 8. T.6 is identical to that of PROJECT-ASYNC and T.7 to that of PROJECT-CE (see [2] Table 6), and T.8 to T.10 to those of [3] Table 6, with the following changes: the #include criterion of T.6 and T.7 (the sketch now includes the headers of the fan and button modules as well as `mymilli.h`), the task count of T.7 (four tasks), and the file lists of T.8 and T.10, which now cover every sketch file, including `fan.h`, `fan.cpp`, `button.h` and `button.cpp`. The button debounce must also use `mymilli()` as its only time source (T.8). The following are added in this revision:

Table 13. Testing and validation requirements added in this revision

ID	Testing requirement	Verifies (HL / EC)
T.11 (added 10/31/26 – RP)	The motor must have a rated voltage inside 3 V to 12 V, the supply must limit the current to under 4 A, and the fan must be smaller than 4 in x 4 in x 3 in. The fan must be driven only by the Timer/Counter1 PWM output on pin 9, a 5 V logic signal. One single-pole, momentary-on, normally-off push button must be connected between pin 10 and ground.	HL.7, HL.9, HL.10, HL.11, HL.12
T.12 (added 10/31/26 – RP)	Starting with the fan off, each push of the button (press and release) must move the fan to LOW, MED, HIGH, MED, LOW and OFF in that order, and the sequence must repeat (Table 12).	HL.14, HL.15, EC.8
T.13 (added 10/31/26 – RP)	The speed of the button has no effect on the asynchronous blinking of the LEDs.	HL.13, EC.8
T.14 (added 10/31/26 – RP)	The motor speed, air flow and sound level of LOW, MED and HIGH must be justified by calculation (CALCULATIONS FOR EC.9).	HL.8, HL.16, HL.17, HL.18, EC.9

REQUIREMENTS TRACEABILITY

Table 14. Requirements traceability — every requirement in [1] Tables 3 and 4, in [2] Table 2, in [3] Table 2 and in Tables 2 and 3 of this document is covered by at least one test. Rows added or changed in this revision are marked (Rev 4).

Requirement	Verified by (T.#)	Method
HL.1 – At least 2 LEDs	T.1	Test
HL.2 – Standard Arduino-compatible board	T.2	Inspection
HL.3 – Communication via IDE serial monitor	T.3	Demonstration
HL.4 – Asynchronous serial use; blinking unaffected	T.3	Test
HL.5 – Blink interval set in msec	T.4	Test
HL.6 – Blink rate constant unless changed	T.1	Test
HL.7 – DC motor fan, 3 V to 12 V (Rev 4)	T.11	Inspection
HL.8 – Fan speed 1500 to 6500 RPM (Rev 4)	T.14	Calculation
HL.9 – Fan draws less than 4 A (Rev 4)	T.11	Inspection, Calculation
HL.10 – PWM signal at most 5 VDC (Rev 4)	T.11	Inspection
HL.11 – Fan smaller than 4 in x 4 in x 3 in (Rev 4)	T.11	Inspection
HL.12 – Single-pole momentary-on push button (Rev 4)	T.11	Inspection
HL.13 – No noticeable effect on LEDs and serial (Rev 4)	T.13	Demonstration

Requirement	Verified by (T.#)	Method
HL.14 – Four speeds (Rev 4)	T.12	Demonstration
HL.15 – Button cycles the speeds (Rev 4)	T.12	Demonstration
HL.16 to HL.18 – Air flow and sound of LOW, MED, HIGH (Rev 4)	T.14	Calculation
EC.1 – Built with IDE 2.3.3+ gcc (ATmega328P)	T.5	Inspection
EC.2 / EC.2.1 – On GitHub; Evaluator can build and run	T.5	Demonstration
EC.3 – LEDs on digital pins 2 & 3	T.6	Inspection
EC.4 – Required dialog ([1] Table 7) works	T.3	Demonstration
EC.5 – Bare metal; no 3rd-party multitasking (Rev 4)	T.6, T.7	Inspection
EC.6 – Round-robin cyclic executive (Rev 4)	T.7	Inspection, Test
EC.7 – mymilli() replaces millis() (Rev 4)	T.8	Inspection, Test
EC.7.1 – Own PWM counter ISR	T.9	Inspection, Test
EC.7.2 – Own register constants (Rev 4)	T.10	Inspection
EC.8 – Dialogs of Table 4 and Table 5 of [5] work (Rev 4)	T.12, T.13	Demonstration
EC.9 – H.16 to H.18 justified by inference (Rev 4)	T.14	Calculation

TEST PROCEDURES

The procedures for T.1 through T.5 are identical to those of PROJECT-ASYNC and are run on the PROJECT-ADDMOTOR build. See [1] TESTING AND VALIDATION section, Table 10. The procedure for T.7 is in [2] Table 6, and the procedures for T.8 to T.10 are in [3] Table 8. The searches for #include in T.6 and T.7 expected no matches in the earlier builds. In this revision the sketch includes headers written by The Developer, so the criterion for that search is changed: those lines are the only #include lines in the .ino file and no library or scheduler header is included. T.8 and T.10 are run on every sketch file, including the fan and button modules, and their searches for names in code ignore comments; T.10 also searches for `TCCR1A`, `TCCR1B`, `OCR1A`, `ICR1`, `PINB`, `PORTB`, `DDRB`, `WGM10` to `WGM13`, `COM1A1`, `CS10` to `CS12` and `TCNT1`, and checks the addresses in Table 9 against [6]. The following are added in this revision:

Table 15. Test procedures added in this revision

Test	Procedure (steps)	Expected result / pass criterion
T.11	Read the motor rating on its label or data sheet. Confirm the motor is wired to the 5 V power module (Table 7) and measure the fan with a ruler, not counting any cage. Read the fan module and confirm the PWM comes from Timer/Counter1 on pin 9. Inspect the push button wiring and the pull-up on pin 10.	Rated voltage inside 3 V to 12 V. Supply limits the current to under 4 A. Fan smaller than 4 in x 4 in x 3 in. PWM on pin 9 at 5 V logic. One single-pole, momentary-on, normally-off button between pin 10 and ground, high released and low pressed.

Test	Procedure (steps)	Expected result / pass criterion
T.12	Power the System with the fan off. Press and release the button six times and record the fan state after each release. Continue for at least one more cycle.	LOW, MED, HIGH, MED, LOW, OFF in that order, with the three speeds clearly different, and the sequence repeats (Table 12).
T.13	Open the Serial Monitor at 9600 baud with the line ending set to Newline. Enter 1 then 500, then 2 at 100. While both LEDs blink, press the button through the full fan sequence at least twice.	LED 2 blinks 300 ms on and 300 ms off, LED 1 800 ms on and 800 ms off, and neither changes while the fan steps. The prompt "What LED? (1 or 2)" returns and the dialog still accepts entries. The fan follows T.12.
T.14	Review CALCULATIONS FOR EC.9.	LOW, MED and HIGH meet HL.8, HL.16, HL.17 and HL.18 under the stated assumptions.

CALCULATIONS FOR EC.9

EC.9 allows H.16, H.17 and H.18 to be justified by inference. The calculations use assumption (1) of DESIGN RATIONALE AND ASSUMPTIONS: speed and air flow scale with the duty cycle.

Current and size (HL.9, HL.11). The fan is the small brushed DC motor with plastic fan blade from the ELEGOO Uno R3 starter kit, rated 3 V to 6 V (inside the 3 V to 12 V of HL.7). It is powered only from the 5 V breadboard power module, whose regulator limits the current to about 700 mA. Motors of this type draw about 70 mA unloaded and about 600 mA with the fan blade, with a stall current near 1.2 A, so the module's limit is the largest current the motor can draw. 0.7 A is below the 4 A limit of HL.9. Fan size: about 2.6 in (blade diameter) x 2.6 in x 1.5 in (blade plus motor depth), under the limit of 4 in x 4 in x 3 in, not including cage.

Speed (HL.8). HL.8 requires the fan to operate at a minimum speed between 1500 and 6500 RPM, so the check is made on LOW, the lowest running state. Let N_HIGH be the speed at the full 5 V. With speed proportional to duty, N_LOW = 0.40 × N_HIGH and N_MED = 0.64 × N_HIGH. N_LOW is inside 1500 to 6500 RPM when 3750 RPM ≤ N_HIGH ≤ 16,250 RPM. Listings for 3 V to 6 V motors of this type give about 10,000 RPM with no load at 6 V; scaled to 5 V this is N_HIGH ≈ 8,300 RPM (an estimate from typical data for this type of motor, not a measurement). This gives N_LOW ≈ 3,300 RPM and N_MED ≈ 5,300 RPM. N_LOW is inside 1500 to 6500 RPM, so HL.8 is met.

Air flow and sound (HL.16, HL.17, HL.18). Air flow is proportional to fan speed (fan affinity law), so Q_LOW = 0.40 × Q_HIGH and Q_MED = 0.64 × Q_HIGH, with Q_HIGH between 0.70 and 1.00 CFM from HL.18. The sound level of a fan changes by about 50 × log10(N2 ÷ N1) dB between two speeds, so LOW is 50 × log10(0.40) = −19.9 dB and MED is 50 × log10(0.64) = −9.7 dB relative to HIGH. Table 16 gives the results.

Table 16. Estimated air flow and sound level of each state

State	Speed ratio (duty)	Air flow for Q_HIGH = 0.70 to 1.00 CFM	Allowed range ± 0.05 CFM	Sound if HIGH is under 50 dB
LOW	0.40	0.28 to 0.40 CFM	0.25 to 0.50 CFM: met	under 30.1 dB (limit 35): met

State	Speed ratio (duty)	Air flow for Q_HIGH = 0.70 to 1.00 CFM	Allowed range ± 0.05 CFM	Sound if HIGH is under 50 dB
MED	0.64	0.45 to 0.64 CFM	0.40 to 0.75 CFM: met	under 40.3 dB (limit 40): met when HIGH is under 49.7 dB
HIGH	1.00	0.70 to 1.00 CFM	0.65 to 1.05 CFM: met	under 50 dB (assumed)

TESTING RESULTS

No Revision 4 results have been recorded yet. Revision 3 results (see [3] Table 9) do not carry over, because the code under test has changed. Entries to be recorded are highlighted.

Table 17. Results of tests

Test	Result (Pass / Fail)	Data / observations
T.1	Pass	On the PROJECT-ADDMOTOR build, both LEDs blinked independently at their set intervals and held a constant rate over more than 10 consecutive cycles with no visible drift, including while the fan was running.
T.2	Pass	ELEGOO Uno R3 (ATmega328P). The sketch, with mymilli, fan and button files, compiled and uploaded from Arduino IDE 2.3.10 without errors.
T.3	Pass	The serial monitor showed the prompts in [1] Table 7 and accepted each entry. While LED 2's interval was being entered, LED 1 kept blinking. Transcript: What LED? (1 or 2) → 2, What interval (in msec)? → 600, What LED? (1 or 2) → 1, What interval (in msec)? → 1600, What LED? (1 or 2).
T.4	Pass	Entered interval 600 ms; measured on-time 600 ms (LED1 ON at 18113 ms, OFF at 18713 ms from the serial timestamps)
T.5	Pass	Repository https://github.com/raheemprovost/CMPE311Project4. Downloaded, compiled in Arduino IDE 2.3.10 and ran on the Uno R3 without errors.
T.6	Pass	LEDs are on digital pins 2 and 3 (pinMode(2, OUTPUT); pinMode(3, OUTPUT);). The only #include lines in the sketch are mymilli.h, fan.h and button.h, all written by The Developer. No multitasking library is included.
T.7	Pass	taskTable[] lists askQuestions, serviceLED1, serviceLED2 and checkButton. loop() only calls taskTable[currentTask]() and advances currentTask modulo NUM_TASKS. No delay(matches. The only #include lines are mymilli.h, fan.h and button.h.
T.8	Pass	Searched the .ino, mymilli.h, mymilli.cpp, fan.h, fan.cpp, button.h and button.cpp for millis(, micros(and delay(in code and found no matches. mymilli() is called by blinkLED1(), blinkLED2() and the button debounce in button.cpp.
T.9	Pass	mymilli.cpp defines its own compare-match ISR for Timer/Counter0 in CTC mode with OCR0A = 249. The sketch built and uploaded with no errors, and the LEDs blink at the entered intervals, so the count is advancing. The fan uses Timer/Counter1 hardware PWM and adds no interrupt.

Test	Result (Pass / Fail)	Data / observations
T.10	Pass	Whole-word search of the code files found no TCCR0A, TCCR0B, OCR0A, TIMSK0, OCIE0A, WGM01, CS00 to CS02, TCNT0, TCCR1A, TCCR1B, OCR1A, ICR1, PINB, PORTB, DDRB, WGM10 to WGM13, COM1A1, CS10 to CS12 or TCNT1. The registers are accessed through the constants in mymilli.cpp, fan.cpp and button.cpp. The Timer/Counter1 and Port B addresses (0x80, 0x81, 0x86, 0x88, 0x23, 0x24, 0x25) match the data sheet.
T.11	Pass	The motor is rated 3 V to 6 V (typical rating for this type of motor), inside 3 V to 12 V. It runs from the 5 V power module, which limits current to about 700 mA, under 4 A. Fan measures about 2.6 in x 2.6 in x 1.5 in (from the listing for a fan of this type; to be confirmed by measurement), under 4 in x 4 in x 3 in. The PWM comes from Timer/Counter1 on pin 9 at a 5 V logic level. One single-pole, momentary-on, normally-off button is wired between pin 10 and ground (reads high released, low pressed).
T.12	Pass	Starting with the fan off, each press and release stepped the fan LOW, MED, HIGH, MED, LOW, OFF in that order and back to LOW again.
T.13	Pass	With LED 2 at 100 ms and LED 1 at 500 ms, times. LEDs are running asynchronously.
T.14	Pass	By calculation (Table 16), with speed and air flow proportional to duty: LOW is 0.28–0.40 CFM, MED is 0.45–0.64 CFM and HIGH is 0.70–1.00 CFM, all inside the allowed ranges. HL.8 is met because N_HIGH is estimated at about 8,300 RPM (typical data for this type of motor, not measured), which gives N_LOW ≈ 3,300 RPM, inside 1500 to 6500 RPM.

APPENDIX A. TEST EVIDENCE

Evidence for T.1 through T.14 is recorded on the PROJECT-ADDMOTOR build. Evidence from Revisions 1, 2 and 3 remains in [1] APPENDIX A, [2] APPENDIX A and [3] APPENDIX A.

Table 18. Index of test evidence

Item	Description	File name / link	Supports test
Video	Both LEDs blinking independently at set intervals, ADDMOTOR build	https://github.com/raheemprovost/CMPE311Project4/blob/main/CMPE311proj4vid.mp4	T.1
Screenshot	Successful IDE upload to the board	https://github.com/raheemprovost/CMPE311Project4/blob/main/CMPE311proj4Upload.png	T.2
Serial Monitor screenshot	Dialog from [1] Table 7 running with both LEDs unaffected	https://github.com/raheemprovost/CMPE311Project4/blob/main/CMPE311proj4SerialMonitor.png	T.3
Video	Measured on/off time vs. entered interval	https://github.com/raheemprovost/CMPE311Project4/tree/main/Proof%20of%20measured%20time	T.4
Link	GitHub repository	https://github.com/raheemprovost/CMPE311Project4	T.5
Code screenshot	Pin setup (pinMode calls) and full #include list	https://github.com/raheemprovost/CMPE311Project4/tree/main/Proof%20of%20no%20pin%20writing%20or%20multitasking	T.6
Code screenshot	taskTable[] and loop() dispatcher	https://github.com/raheemprovost/CMPE311Project4/tree/main/Proof%20of%20tasktable%20and%20dispatcher	T.7
Code screenshot	Searches of all sketch files for millis(, micros(and delay(with no matches; mymilli() calls in the blink functions and the debounce	https://github.com/raheemprovost/CMPE311Project4/tree/main/Proof%20of%20no%20predefined%20functions	T.8
Code and build screenshot	Compare-match ISR, timer configuration and the clean build output	https://github.com/raheemprovost/CMPE311Project4/tree/main/Proof%20of%20compare%20match%20and%20timer%20configuration	T.9
Code screenshot	Whole-word search for AVR timer and port names with no matches; register constant blocks	https://github.com/raheemprovost/CMPE311Project4/tree/main/Register%20constants%20block	T.10
Photo	Fan, motor label, measured fan size and push button wiring	https://github.com/raheemprovost/CMPE311Project4/blob/main/ProofofHardware.png	T.11
Video	Fan sequence of Table 12, repeated	https://github.com/raheemprovost/CMPE311Project4/blob/main/CMPE311proj4fanvid.mp4	T.12

Item	Description	File name / link	Supports test
Video	LED dialog with the button pressed through the fan sequence	https://github.com/raheemprovost/CMPE311Project4/blob/main/CMPE311proj4vid.mp4	T.13
Calculation	CALCULATIONS FOR EC.9 in this document (Table 16)	This document	T.14

END OF DOCUMENT